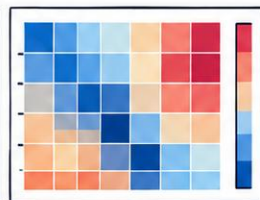
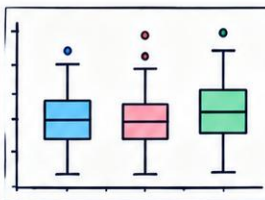
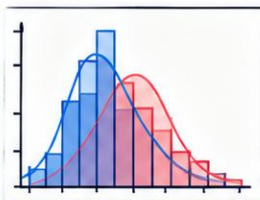
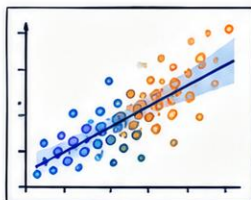


Simple
Visualizations.
Deeper
Insights!

Same Data.
More Beautiful
Visuals!

Seaborn

FULL NOTES



Your Complete Guide to Seaborn

From
Exploration
to Insights!

— DataXOdyssey —

Learn.
Practice.
Visualize.
Grow!

Seaborn Fundamentals

Beautiful
Statistical
Visualizations
in Python

1. What is Seaborn ?

- Seaborn is a powerful data visualization library built on top of Matplotlib.
- Provides a high-level interface for drawing attractive & informative statistical graphics.
- Works seamlessly with Pandas data structures.
- Makes complex visualizations easier with fewer lines of code.

2. Installation

Install using pip :

```
pip install seaborn
```

Seaborn is usually installed with Matplotlib, NumPy and Pandas.

3. Importing Seaborn

```
import seaborn as sns
import matplotlib.pyplot as plt
```

4. Built-in Datasets

- Seaborn comes with several sample datasets for quick exploration.
- Useful for learning and testing visualizations.

Example :

```
sns.get_dataset_names()
```

Some popular datasets :

- tips
- iris
- titanic
- flights
- penguins
- diamonds

5. Setting the Theme

- Seaborn provides beautiful default styles to make plots look better automatically.
- Use `sns.set_theme()` to control the overall style.

```
sns.set_theme(style="whitegrid",
              palette="deep",
              font_scale=1.1)
```

Common styles:

- darkgrid (default)
- whitegrid
- dark
- white
- ticks

6. Figure and Axes Basics

- Seaborn works with Matplotlib's Figure and Axes.
- You can create a figure and axes using `plt.subplots()`.

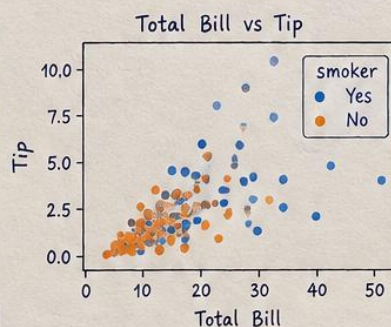
```
fig, ax = plt.subplots(figsize=(6,4))
sns.scatterplot(x="total_bill",
               y="tip",
               data=sns.load_dataset("tips"),
               ax=ax)
```

fig → whole figure
ax → individual plot (axes)

7. A Quick Example

- Let's create a simple plot using a built-in dataset.

```
import seaborn as sns
import matplotlib.pyplot as plt
sns.set_theme()
df = sns.load_dataset("tips")
sns.scatterplot(x="total_bill",
               y="tip",
               hue="smoker",
               data=df)
plt.title("Total Bill vs Tip")
plt.show()
```



Key Takeaways

- ✓ Seaborn simplifies data visualization.
- ✓ Built on Matplotlib.
- ✓ Comes with sample datasets.
- ✓ Use `sns.set_theme()` for beautiful defaults.
- ✓ Works well with Pandas DataFrames.
- ✓ Start with simple plots and explore from there!

Why Use Seaborn ?

- ★ Less code, more beautiful plots
- ★ Built-in datasets
- ★ Great for statistical visualizations
- ★ Easy integration with Pandas
- ★ Ideal for Exploratory Data Analysis (EDA)

"Turn your data into insights, with just a few lines of code!"

Next Up

- 👉 Explore different types of Seaborn plots like `scatterplot()`, `lineplot()`, `barplot()` and more!

DataXOdyssey

Learn Visualize Grow

Seaborn - Basic Plots

Visualize Your Data with Just a Few Lines of Code !

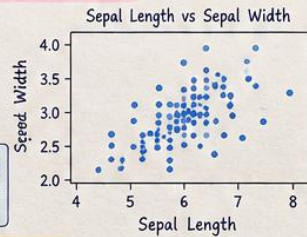
Simple Plots
Powerful Insights

1. Scatter Plot (scatterplot)

- Shows the relationship between two numerical variables.
- Helps identify patterns, trends and outliers.

```
sns.scatterplot(x="sepal_length",  
y="sepal_width",  
data=df)
```

Use hue= to add color categories.

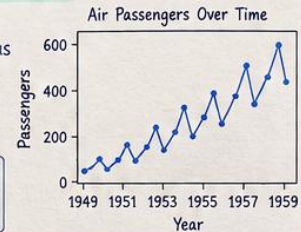


2. Line Plot (lineplot)

- Shows the trend of a variable over a continuous range (e.g., time).
- Useful for time series data.

```
sns.lineplot(x="year",  
y="passengers",  
data=df)
```

Can also show confidence intervals (default).

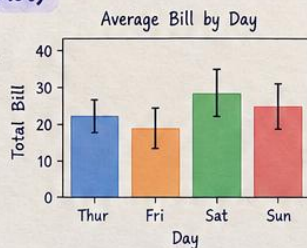


3. Bar Plot (barplot)

- Shows the aggregated value (default: mean) for each category.
- Useful for comparing different groups.

```
sns.barplot(x="day",  
y="total_bill",  
data=df)
```

Use estimator= to change the aggregation (e.g., sum, median).

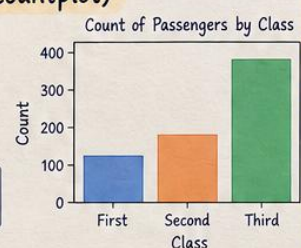


4. Count Plot (countplot)

- Shows the count of observations in each category.
- Useful for categorical data analysis.

```
sns.countplot(x="class",  
data=df)
```

Great for understanding category distribution.

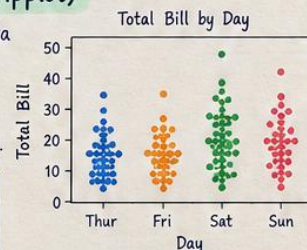


5. Strip Plot (stripplot)

- Shows all individual data points for each category (with slight jitter).
- Useful to see the actual distribution of data points.

```
sns.stripplot(x="day",  
y="total_bill",  
data=df)
```

Use jitter=True (default) to avoid overlapping points.

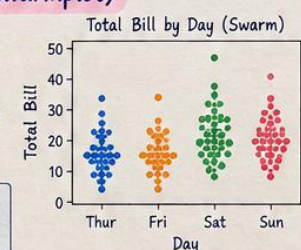


6. Swarm Plot (swarmplot)

- Similar to stripplot but arranges points without overlap.
- Gives a clearer view of distribution (better for smaller datasets).

```
sns.swarmplot(x="day",  
y="total_bill",  
data=df)
```

Useful when you want to see every data point clearly.

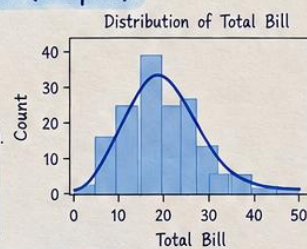


7. Distribution Plot (histplot)

- Shows the distribution of a numerical variable using a histogram.
- Helps understand the shape (skewness, spread etc.).

```
sns.histplot(x="total_bill",  
data=df,  
kde=True)
```

Use kde=True to show the density curve.

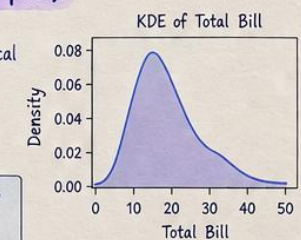


8. KDE Plot (kdeplot)

- Shows the smooth distribution of a numerical variable using Kernel Density Estimation.
- Useful for comparing distributions.

```
sns.kdeplot(x="total_bill",  
data=df,  
fill=True)
```

Set fill=True to fill the area under the curve.



Key Takeaways

- ✓ Seaborn makes it easy to create beautiful statistical plots.
- ✓ Use the right plot for the right type of data (numerical / categorical).
- ✓ Customize with hue, style, size, palette, etc. for more insights.
- ✓ Just a few lines of code, big visual impact !

"A picture with data
is worth a thousand insights!"

DataXOdyssey

Learn Visualize Grow

Seaborn - Distribution Plots

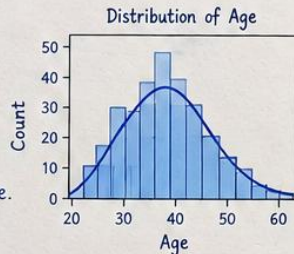
Understand the Distribution of Your Data !

Explore Patterns
Identify Outliers
Gain Insights

1. histplot()

- Shows the distribution of a numerical variable using a histogram.
- Useful to understand the shape, spread and frequency of data.
- Can also add a KDE curve.

```
sns.histplot(x="age",  
             data=df,  
             kde=True,  
             bins=20)
```

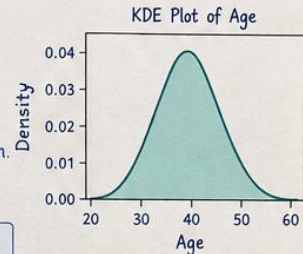


Helps you see the overall distribution at a glance.

2. kdeplot()

- Shows a smooth probability density curve (Kernel Density Estimation).
- Useful to understand the underlying distribution.
- Can be used alone or with histplot().

```
sns.kdeplot(x="age",  
            data=df,  
            fill=True,  
            color="teal")
```

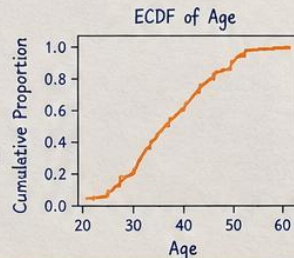


A smooth curve that shows the probability density.

3. ecdfplot()

- Shows the Empirical Cumulative Distribution Function (ECDF).
- Useful to understand the cumulative proportion of data.

```
sns.ecdfplot(x="age",  
             data=df)
```

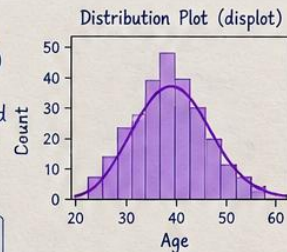


Shows the proportion of values less than or equal to a given value.

4. displot()

- A figure-level function that combines histplot() and FacetGrid.
- Useful for more advanced distributions and multi-plot layouts.
- Good for quick EDA.

```
sns.displot(x="age",  
            data=df,  
            kde=True,  
            color="purple")
```

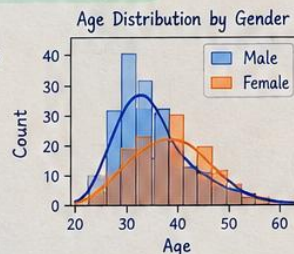


A high-level function for easier distribution plots.

5. Multiple Distributions (hue)

- Compare distributions across different categories using the hue parameter.

```
sns.histplot(x="age",  
             hue="gender",  
             data=df,  
             kde=True,  
             multiple="stack")
```

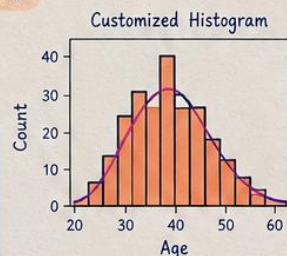


Great for comparing distributions across groups.

6. Customizing Plots

- You can customize bins, color, kde, etc. to make your plots more informative.

```
sns.histplot(x="age",  
             data=df,  
             bins=15,  
             kde=True,  
             color="coral",  
             edgecolor="black")
```



Customize to make your visualizations more insightful !

When to Use What ?

- histplot() → Frequency distribution (histogram)
- kdeplot() → Smooth density curve
- ecdfplot() → Cumulative distribution
- displot() → Quick & advanced distribution plots
- hue → Compare groups
- Customize → Better insights

Key Takeaways

- ✓ Seaborn makes it easy to understand the distribution of your data.
- ✓ Use histplot() for a quick view.
- ✓ Use kdeplot() for smooth density.
- ✓ Use ecdfplot() for cumulative insights.
- ✓ Use displot() for advanced layouts.
- ✓ Compare groups using hue.
- ✓ Customize plots to tell a better story.

Real-World Use Cases

- ★ Understand customer age distribution
- ★ Analyze income or salary spread
- ★ Detect outliers
- ★ Compare feature distributions across groups
- ★ Perform exploratory data analysis (EDA)

"Data tells a story,
distribution shows the big picture."

DataXOdyssey

Learn Visualize Grow

Seaborn - Relationship Analysis

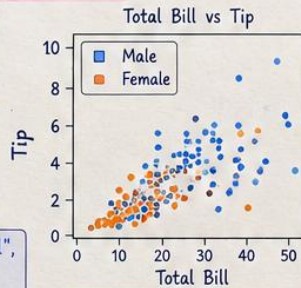
Explore the Relationship Between Variables!

Find Patterns
Trends
Correlations
in Your Data!

1. Scatter Plot (scatterplot)

- Shows the relationship between two numerical variables.
- Helps identify patterns, trends, clusters and outliers.
- Use hue, style, size to add more information.

```
sns.scatterplot(x="total_bill",
                y="tip",
                data=df,
                hue="sex")
```

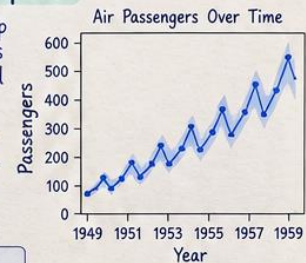


As total bill increases, tip tends to increase.

2. Line Plot (lineplot)

- Shows the relationship between two variables as a continuous trend (e.g., over time).
- Useful for time series data or ordered data.
- Can show confidence intervals (default).

```
sns.lineplot(x="year",
              y="passengers",
              data=flights)
```

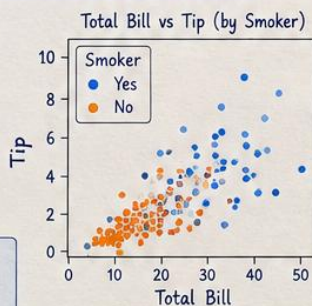


Shows an increasing trend with seasonal patterns.

3. relplot() - Flexible Relationship Plot

- A figure-level function for relationship plots.
- Can create scatter or line plots using kind parameter.
- Easily create faceted plots with row, col and hue.

```
sns.relplot(x="total_bill",
            y="tip",
            hue="smoker",
            style="sex",
            size="size",
            data=df, kind="scatter")
```

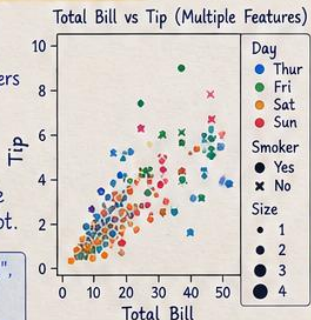


Easily add multiple dimensions to your plot.

4. Using hue, style and size

- hue → color based on a category.
- style → different markers for categories.
- size → vary the size of points.
- Helps visualize multiple variables in a single plot.

```
sns.scatterplot(x="total_bill",
                y="tip",
                hue="day",
                style="smoker",
                size="size",
                data=df)
```

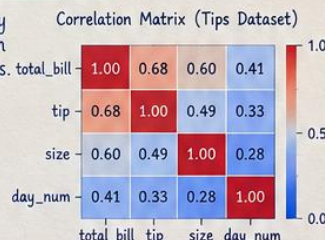


Visualize multiple variables at once!

5. Correlation with heatmap

- Use a heatmap to quickly see the correlation between multiple numerical variables.
- Helps identify strong positive or negative relationships.

```
sns.heatmap(df.corr(),
            annot=True,
            cmap="coolwarm")
```

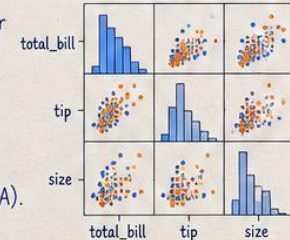


Darker red → stronger positive correlation
Darker blue → stronger negative correlation

6. pairplot() - All Relationships at Once

- Creates a grid of scatter plots for all numerical variables in the dataset.
- Also shows distribution plots on the diagonal.
- Useful for quick exploratory analysis (EDA).

```
sns.pairplot(df,
              hue="smoker")
```



Quickly explore relationships between multiple variables.



Key Takeaways

- Use scatterplot() for simple relationships.
- Use lineplot() for trends over time.
- Use relplot() for flexible, faceted plots.
- Use hue, style and size to add more information.
- Use heatmap() to check correlations.
- Use pairplot() for quick EDA of multiple variables.

"Relationships in data reveal stories that numbers alone cannot."

Real-World Use Cases

- Analyze sales vs advertising spend
- Study customer behavior
- Understand feature relationships in ML datasets
- Identify correlated variables
- Perform exploratory data analysis (EDA)

DataXOdyssey

Learn Visualize Grow

Seaborn - Statistical Visualization

Go beyond simple plots – uncover the statistics in your data!

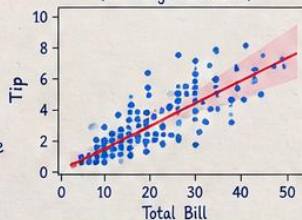
Visualize Trends
Uncertainty
Relationships
with Ease!

1. Understanding Statistical Plots

- Seaborn provides built-in functions to visualize statistical relationships in data.
- Helps to see trends, uncertainty and patterns easily.
- Built on top of Matplotlib with a clean and attractive style.
- Common statistical plots: regression, confidence intervals, distributions etc.

Less code, more insights!

Example Dataset: tips
Total bill vs Tip
(with regression line)



```
import seaborn as sns
tips = sns.load_dataset("tips")
sns.regplot(x="total_bill",
            y="tip", data=tips)
```

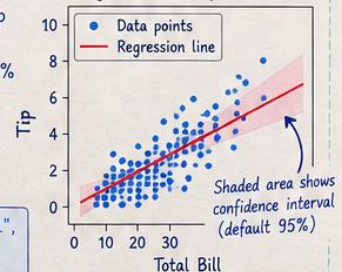
2. Regression Plot (regplot)

- Fits a regression line to show the relationship between two variables.
- By default, shows a 95% confidence interval.
- Useful to understand trends and make predictions.

```
sns.regplot(x="total_bill",
            y="tip", data=tips)
```

Great for quick analysis of relationships.

Regression Plot (Tips Dataset)



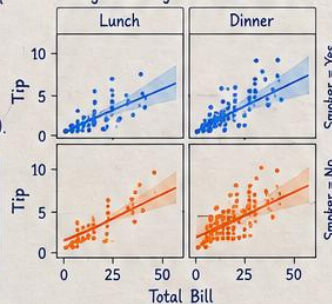
3. Linear Model Plot (lplot)

- Combines regplot() with FacetGrid.
- Useful for visualizing regression across multiple categories (e.g., by gender or smoker).

```
sns.lmplot(x="total_bill",
           y="tip",
           hue="smoker",
           col="time",
           data=tips)
```

Easily compare trends across groups.

Regression by Time and Smoker

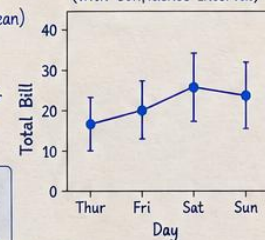


4. Estimator and Confidence Interval

- Many Seaborn plots (like lineplot, pointplot, barplot) show an estimate (default: mean) with a confidence interval (default: 95%).
- You can change the estimator and disable the confidence interval if needed.

```
sns.lineplot(x="day",
             y="total_bill",
             data=tips,
             estimator="mean",
             errorbar="sd")
```

Average Bill by Day
(with Confidence Interval)



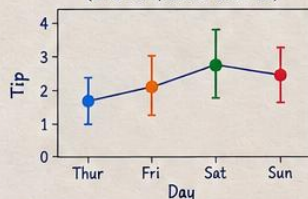
Shows the mean total bill with variability across days.

5. Categorical Statistical Plots

- Visualize the distribution of a numeric variable across categories.
- Common plots: barplot(), pointplot(), violinplot(), boxplot().

```
sns.pointplot(x="day",
              y="tip",
              data=tips,
              errorbar="ci")
```

Average Tip by Day
(with Confidence Interval)



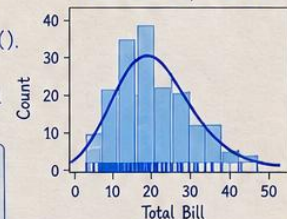
Point plot shows the mean tip with 95% confidence interval.

6. Visualizing Distributions with Stats

- Combine distribution and statistical information using kdeplot() or histplot().
- You can also add a rug plot to show individual observations.

```
sns.histplot(tips["total_bill"],
             kde=True,
             rug=True,
             bins=20)
```

Distribution of Total Bill



Histogram + KDE + Rug plot gives a complete view of the distribution.



Key Takeaways

- ✓ Use regplot() for simple regression.
- ✓ Use lmplot() for multi-panel analysis.
- ✓ Understand and customize estimator and confidence intervals.
- ✓ Use categorical plots to compare groups.
- ✓ Combine distribution plots with statistical info.
- ✓ Seaborn makes statistical visualization simple and visually appealing.

"Statistics turn data into stories, and Seaborn helps you tell them beautifully!"

DataXOdyssey

Real-World Use Cases

- ★ Analyze sales trends over time.
- ★ Compare performance across regions.
- ★ Study the effect of features on target variable (e.g., price vs size).
- ★ Understand customer behavior across segments.
- ★ Perform exploratory data analysis (EDA).
- ★ Build data-driven insights for reports and dashboards.

Learn Visualize Grow

Seaborn - Heatmaps & Matrix Plots

Visualize Correlations, Patterns and Relationships in a Matrix Form!

Turn Numbers into Meaningful Patterns!

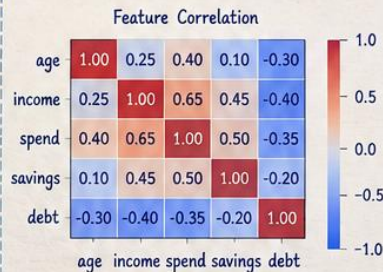
1. What is a Heatmap?

- A heatmap displays data in a matrix form using colors.
- Useful to see patterns, correlations and outliers quickly.
- Commonly used for numerical data.

Darker / warmer colors usually indicate higher values.

2. Basic Heatmap

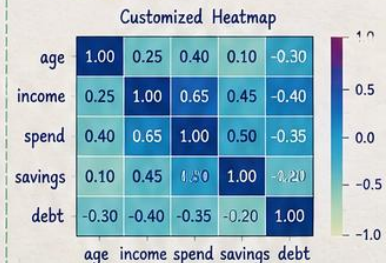
Visualize a correlation matrix.



```
sns.heatmap(df.corr(),
             annot=True,
             cmap="coolwarm")
```

3. Customizing Heatmap

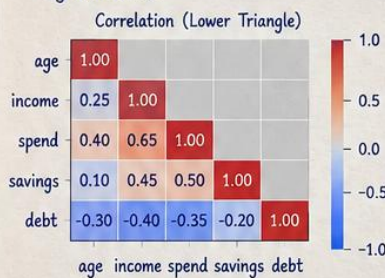
- Add annotations, change color map, adjust figure size, add labels, etc.



```
sns.heatmap(df.corr(), annot=True,
             cmap="YlGnBu", fmt=".2f",
             linewidths=0.5, linecolor="white")
```

4. Masking the Heatmap

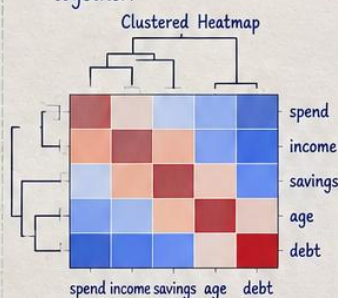
- Use a mask to hide one half of the matrix (since correlation matrices are symmetric).



```
mask = np.triu(np.ones_like(df.corr(), dtype=bool))
sns.heatmap(df.corr(), annot=True, mask=mask,
             cmap="coolwarm", fmt=".2f")
```

5. clustermap()

- Creates a clustered heatmap with hierarchical clustering.
- Helps group similar features together.

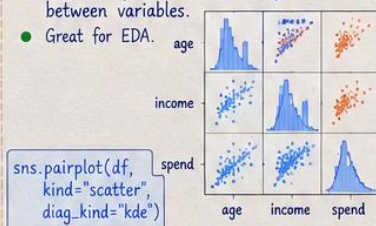


```
sns.clustermap(df.corr(),
               cmap="coolwarm",
               annot=True, fmt=".2f")
```

6. Other Matrix Plots

Pairplot

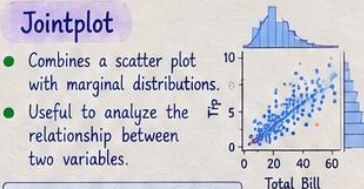
- Visualize pairwise relationships between variables.
- Great for EDA.



```
sns.pairplot(df,
             kind="scatter",
             diag_kind="kde")
```

Jointplot

- Combines a scatter plot with marginal distributions.
- Useful to analyze the relationship between two variables.



```
sns.jointplot(x="total_bill",
              y="tip", kind="scatter")
```

7. Choosing the Right Plot

Plot	Use Case
heatmap()	Visualize a matrix (e.g., correlation) with color intensity
clustermap()	Cluster and visualize patterns
pairplot()	Pairwise relationships for multiple variables
jointplot()	Relationship between two variables with marginal plots

8. Key Takeaways

- Heatmaps are great for seeing patterns in numerical data.
- Use annotations for clarity.
- Use masking to remove redundant information.
- clustermap() helps find groups of similar features.
- Pairplot() and jointplot() are useful for EDA.

9. Real-World Use Cases

- Feature correlation in ML datasets
- Customer segmentation analysis
- Identifying multicollinearity between features
- Exploratory data analysis (EDA)
- Visualizing similarity matrices (e.g., in recommender systems)
- Financial and sales data analysis

"Patterns are hidden in data
Heatmaps help you see them!"

DataXOdyssey

Learn Visualize Grow

Seaborn - Multi-Plot Grids

Create Multiple Plots, Discover More Insights !

Same Data
Different Views
Bigger Insights !

1. Why Multi-Plot Grids ?

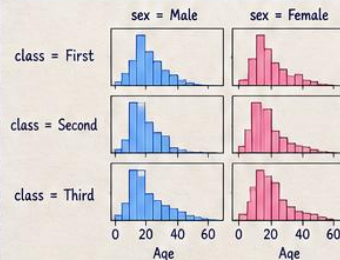
- Visualize multiple relationships at once.
- Compare across different categories or features.
- Helps in exploratory data analysis (EDA).
- Makes it easy to spot patterns and outliers.

Same data, multiple perspectives
= deeper understanding!

2. FacetGrid

- The base class for creating grid of plots.
- Most flexible but requires more code.
- Useful when you want full control over the plot type and layout.

```
g = sns.FacetGrid(df, col="sex",
                  row="class")
g.map(sns.histplot, "age",
      bins=20)
```

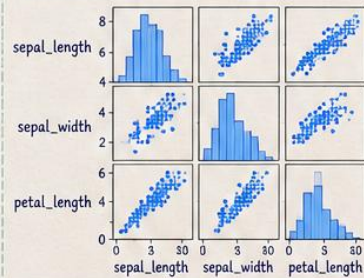


Grid of histograms to compare age distribution by sex and class.

3. PairGrid

- Creates a grid to visualize pairwise relationships in a dataset.
- More flexible than pairplot (you can customize each plot).
- Useful for numerical variables.

```
g = sns.PairGrid(df[["sepal_length",
                    "sepal_width", "petal_length"]])
g.map_diag(sns.histplot, kde=True)
g.map_offdiag(sns.scatterplot)
```

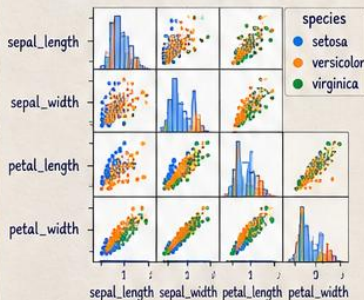


Visualize pairwise relationships between multiple numerical variables.

4. pairplot()

- A high-level function (built on PairGrid).
- Automatically creates pairwise plots for all numerical variables.
- Can also include hue for categorical coloring.

```
sns.pairplot(df, hue="species",
             diag_kind="kde")
```



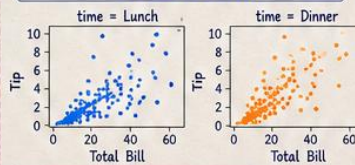
Quickly explore relationships across all features with category information.

5. Col and Row Faceting

- Use col and row to split plots based on a categorical variable.
- Helps compare distributions or trends across subgroups.

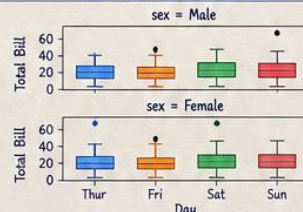
Example 1: Column Facet (col)

```
sns.relplot(data=df, x="total_bill",
            y="tip", col="time",
            kind="scatter")
```



Example 2: Row Facet (row)

```
sns.catplot(data=df, x="day", y="total_bill",
            row="sex", kind="box")
```



Compare distributions across categories.

6. Customizing Grids

- You can customize figure size, color, style, markers, etc.
- Pass common arguments like height, aspect, palette, hue, etc.

```
sns.relplot(data=df, x="total_bill",
            y="tip", hue="smoker",
            col="time", kind="scatter",
            height=3, aspect=1.2,
            palette="Set2")
```

Customize your grid to make it more informative and visually appealing.

7. When to Use What ?

Function	Best For	Flexibility
FacetGrid	Custom grid of plots	High
PairGrid	Pairwise relationships with full control	High
pairplot()	Quick pairwise plots (numerical variables)	Medium
catplot()	Categorical data (facet + plot in one)	Medium
relplot()	Relational data (facet + plot in one)	Medium



Key Takeaways

- ✓ Use FacetGrid for full control.
- ✓ Use PairGrid or pairplot() for pairwise analysis.
- ✓ Use col and row to compare subgroups.
- ✓ Customize with hue, palette, height, aspect, etc.
- ✓ Multi-plot grids make EDA faster and more insightful.

DataXOdyssey

"Multiple views,
clearer stories!"

Learn Visualize Grow

Seaborn - Styling & Customization

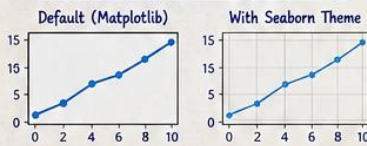
Make Your Visualizations More Beautiful and Meaningful!

Same Plot
Different Styles
Endless Possibilities!

1. Setting the Theme

- Seaborn comes with built-in themes to control the overall look and feel.
- Use `sns.set_theme()` for a consistent style across all plots.
- You can also set style, context and color palette.

```
sns.set_theme(style="whitegrid",  
             context="notebook",  
             palette="deep")
```

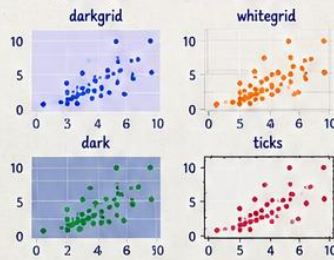


Same data, a cleaner and more modern look!

2. Style Options

- Control the background style of the plot.
- Options: "darkgrid", "whitegrid", "dark", "white", "ticks".

```
sns.set_style("darkgrid")
```

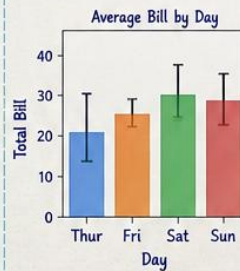


Choose a style that fits your data story.

3. Context (Scale Elements)

- Controls the size of plot elements (text, labels, ticks, etc.).
- Options: "paper", "notebook", (default), "talk", "poster".

```
sns.set_context("talk")
```



Use a larger context for presentations and a smaller one for reports/notebooks.

4. Color Palettes

- Seaborn provides beautiful color palettes.
- Use palette parameter or `sns.set_palette()`.

```
sns.color_palette("Set2")
```



```
sns.set_palette("viridis")
```

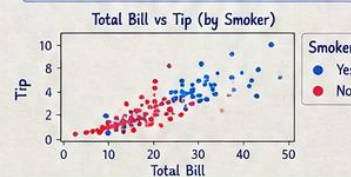


Use the right palette to make your plots more clear and visually appealing.

5. Customizing Colors with hue

- Use `hue` to add color based on a categorical variable.
- You can also choose a specific palette.

```
sns.scatterplot(data=df, x="total_bill", y="tip", hue="smoker",  
               palette="Set1")
```

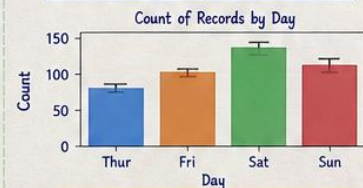


Color adds another dimension to your data.

6. Figure Size & Layout

- Control the size of the figure using `plt.figure()` or `figsize`.
- Use `tight_layout()` to adjust spacing.

```
plt.figure(figsize=(8, 4))  
sns.countplot(data=df, x="day")  
plt.tight_layout()
```

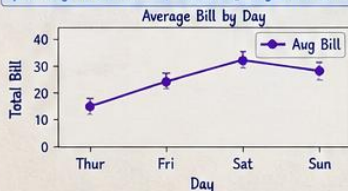


Adjust figure size for better readability.

7. Titles, Labels and Legends

- Use `plt.title()`, `plt.xlabel()`, `plt.ylabel()`.
- Customize legends using `plt.legend()`.
- Remove legend with `legend=False`.

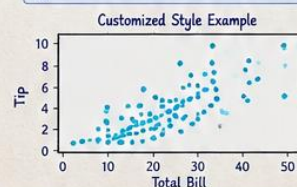
```
sns.lineplot(data=df, x="day", y="total_bill",  
            marker="o")  
plt.title("Average Bill by Day")  
plt.xlabel("Day")  
plt.ylabel("Total Bill")  
plt.legend(title="", labels=["Avg Bill"])
```



8. Using Custom Styles

- You can create your own style or use seaborn's predefined styles.

```
sns.set_style("ticks", {  
    "axes.facecolor": "#f7f7f7",  
    "grid.color": "#d3d3d3",  
    "axes.edgecolor": "#333333"  
})
```

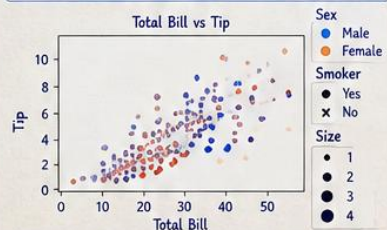


Customize styles to match your personal or brand aesthetic.

9. Additional Customizations

- Control marker style, line style, transparency and more.

```
sns.scatterplot(data=df, x="total_bill", y="tip",  
               hue="sex", style="smoker", size="size",  
               palette="coolwarm", alpha=0.8)
```



Combine multiple customizations to make your plots uniquely yours!



Key Takeaways

- Use `set_theme()`, `set_style()` and `set_context()` for a consistent look.
- Choose the right color palette.
- Customize titles, labels and legends.
- Adjust figure size and layout for readability.
- Use `hue`, `style` and `size` to add more information.
- Don't be afraid to experiment with your own style!

"Good visuals
turn data into
insights!"

DataX Odyssey

Real-World Use Cases

- Create presentation-ready visualizations
- Build branded dashboards and reports
- Customize plots for business storytelling
- Analyze and compare multiple categories
- Make EDA plots more insightful and appealing
- Adapt plots for different audiences (technical vs non-technical)

Learn Visualize Grow

Cleaner Data
+
Better Visuals
=
Deeper Insights!

Seaborn + Pandas

A Powerful Combo for Exploratory Data Analysis!

Pandas for
Data Handling
Seaborn for
Beautiful Insights!

1. Why Use Seaborn with Pandas?

- Pandas makes it easy to load, clean and manipulate data.
- Seaborn works seamlessly with Pandas DataFrames.
- You can directly pass a DataFrame to Seaborn functions.
- Less code, more insights!

2. Load and Explore Data

```
import pandas as pd
import seaborn as sns

# Load dataset
df = sns.load_dataset("tips")

# Explore the data
df.head()
df.info()
df.describe()
df.columns
```

Seaborn's built-in datasets return Pandas DataFrames.

df.head()					
total_bill	tip	sex	smoker	day	
16.99	1.01	Female	No	Sun	
10.34	1.66	Male	No	Sun	
21.01	3.50	Male	No	Sun	
23.68	3.31	Male	No	Sun	
24.59	3.61	Female	No	Sun	
...	...	...	...	...	

Notice: It's a Pandas DataFrame – easy to work with and ready to visualize!

3. Select and Filter Data

- Use Pandas to filter and prepare data before plotting.

```
# Filter only dinner data
dinner_df = df[df["time"] == "Dinner"]

# Select specific columns
df_subset = df[["total_bill", "tip", "sex", "smoker"]]
```

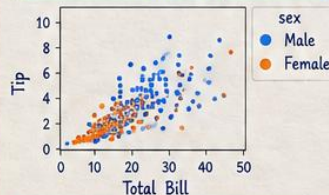
Prepare your data with Pandas, then visualize with Seaborn.

4. Quick Visualizations

- Pass your Pandas DataFrame directly to Seaborn.

```
import seaborn as sns

# Scatter plot
sns.scatterplot(data=df,
                x="total_bill", y="tip",
                hue="sex")
```



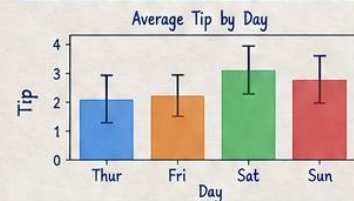
No need to extract columns separately – just use the column names!

5. Group and Visualize

- Use Pandas groupby for aggregation and Seaborn for plotting.

```
# Average tip by day
tip_by_day = df.groupby("day")["tip"].mean()
print(tip_by_day)

# Visualize
sns.barplot(data=df, x="day", y="tip",
            estimator="mean", errorbar="sd")
```

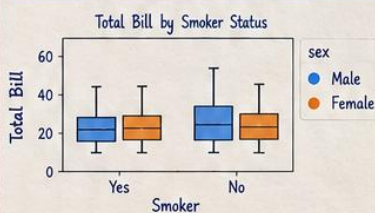


Pandas helps you summarize, Seaborn brings it to life!

6. Use with Categorical Data

- Pandas handles categories easily and Seaborn visualizes them beautifully.

```
# Boxplot of total bill by smoker status
sns.boxplot(data=df, x="smoker",
            y="total_bill", hue="sex")
```

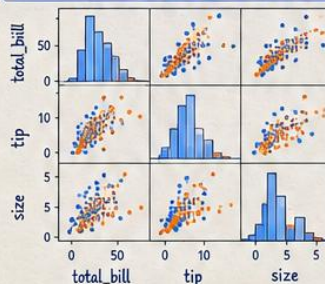


Works directly with Pandas categorical columns (no extra preprocessing needed).

7. Multiple Variables in One Plot

- Use Pandas + Seaborn to explore relationships across variables.

```
sns.pairplot(df,
             hue="sex",
             vars=["total_bill", "tip", "size"])
```



Quickly spot trends, distributions and correlations across multiple variables.

8. Real-World Workflow

A typical EDA flow using Pandas + Seaborn.

- 1 Load data (Pandas)
- 2 Explore and clean data (Pandas)
- 3 Filter or aggregate data (Pandas)
- 4 Visualize with Seaborn
- 5 Iterate and gain insights!



Key Takeaways

- ✓ Seamless integration with Pandas.
- ✓ Less code, more visualization power.
- ✓ Use Pandas for data preparation and Seaborn for insights.
- ✓ Works great for EDA and reporting.
- ✓ Ideal for real-world data analysis.

"Clean data
Beautiful plots
Better decisions!"

DataXOdyssey

Explore Analyze Visualize Grow

Real Data
Real Questions
Real Insights!

Seaborn - Real-World Visualization

From Data to Insights - Visualizations that Tell a Story!

Same Data
Different Visuals
Deeper Understanding!

1. Sales Trend Over Time (Line Plot)

- Question: How have sales changed over time?
- Use Case: Business performance tracking, seasonality analysis.

```
sns.lineplot(data=df, x="date", y="sales",
             hue="category", marker="o")
```

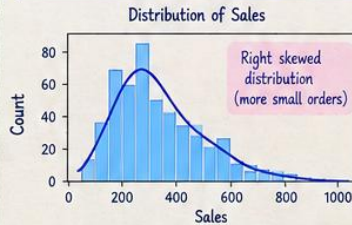


Helps identify trends, seasonality and top performing categories.

2. Sales Distribution (Histogram + KDE)

- Question: What is the distribution of sales?
- Use Case: Understand the spread, skewness and outliers.

```
sns.histplot(df["sales"], kde=True,
             bins=30, color="skyblue")
```



Useful to understand the data distribution before further analysis.

3. Sales vs Profit (Scatter Plot)

- Question: Is there a relationship between sales and profit?
- Use Case: Identify patterns, outliers and high-value segments.

```
sns.scatterplot(data=df, x="sales", y="profit",
                hue="category", style="region",
                s=80, alpha=0.7)
```

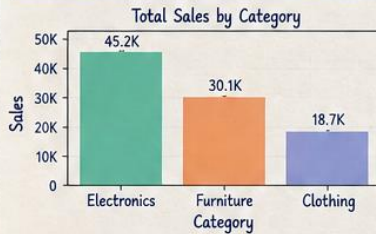


Helps find correlation and spot unusual transactions (e.g., high sales but low profit).

4. Category Performance (Bar Plot)

- Question: Which category has the highest sales?
- Use Case: Compare performance across categories.

```
sns.barplot(data=df, x="category", y="sales",
            estimator="sum", errorbar=None,
            palette="Set2")
```

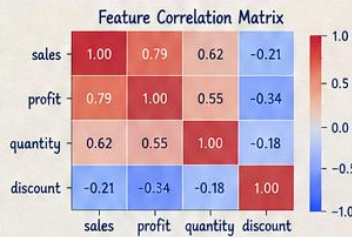


Quickly compare business metrics across categories.

5. Correlation Analysis (Heatmap)

- Question: How are the numeric variables related?
- Use Case: Find strong correlations for feature selection / EDA.

```
sns.heatmap(df.corr(numeric_only=True),
            annot=True, cmap="coolwarm", fmt=".2f")
```

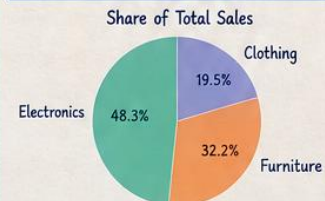


Helps identify relationships between variables (e.g., sales and profit are strongly correlated).

6. Share of Sales (Pie Chart)

- Question: What is the share of each category in total sales?
- Use Case: Visualize proportion of a whole.

```
category_sales = df.groupby("category")["sales"].sum()
sns.set_palette("Set2")
plt.pie(category_sales, labels=category_sales.index,
        autopct="%1.1f%%", startangle=90)
plt.title("Share of Total Sales")
```

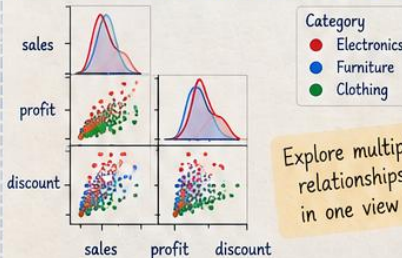


Useful for a quick, high-level understanding of contribution.

7. Multi-Plot Dashboard (PairGrid)

- Question: How do multiple variables relate to each other?
- Use Case: Explore relationships and distributions together.

```
sns.pairplot(df, vars=["sales", "profit", "discount"],
            hue="category", corner=True, palette="Set1")
```



8. Practical Workflow

- 1 Load and explore data (with Pandas)
- 2 Clean and prepare data (if needed)
- 3 Choose the right plot for your question
- 4 Customize (labels, colors, titles)
- 5 Interpret and derive insights
- 6 Combine multiple plots for a complete story

"Visualizations turn data into decisions!"



Key Takeaways

- ✓ Use the right plot for the right question.
- ✓ Seaborn + Pandas makes visualization easy and powerful.
- ✓ Explore distributions, relationships and comparisons.
- ✓ Customize your plots to make them clear and meaningful.
- ✓ Combine multiple plots for deeper insights.
- ✓ Beautiful visuals help you communicate findings effectively.

Real-World Applications

- ★ Business & sales analytics
- ★ Customer segmentation
- ★ Marketing campaign performance
- ★ Financial data analysis
- ★ Product performance comparison
- ★ Exploratory Data Analysis (EDA)
- ★ Building dashboards and reports

Same Data.
Different Questions.
Infinite Insights!

DataXOdyssey

Learn Visualize Grow

Better
Plots
Better Insights!

Seaborn - Best Practices

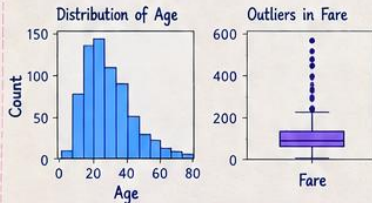
Create Clearer, More Meaningful and Effective Visualizations!

Same Data
Better Visuals
Real Impact!

1. Know Your Data First

- Understand data types, distributions and relationships before plotting.
- Check for missing values and outliers.
- Choose the right plot for your question.

```
df.info()
df.describe()
df.isnull().sum()
```



Good visualizations start with a good understanding of your data.

2. Choose the Right Plot

- Use plots that match your goal (e.g., distribution, comparison, relationship, composition).
- Avoid unnecessary complexity.

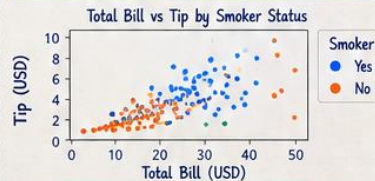
Goal	Recommended Plot
Distribution	histplot(), kdeplot()
Compare categories	barplot(), boxplot(), violinplot()
Relationship	scatterplot(), lineplot()
Correlation	heatmap()
Part-to-whole	countplot(), pie chart (matplotlib)
Multiple variables	pairplot(), FacetGrid()

Right plot = clearer story + better insights!

3. Use Meaningful Labels & Titles

- Add clear titles, axis labels and legends.
- Use descriptive names (avoid generic x, y).
- Make your plots self-explanatory.

```
sns.scatterplot(data=df, x="total_bill", y="tip",
                hue="smoker")
plt.title("Total Bill vs Tip by Smoker Status")
plt.xlabel("Total Bill (USD)")
plt.ylabel("Tip (USD)")
```

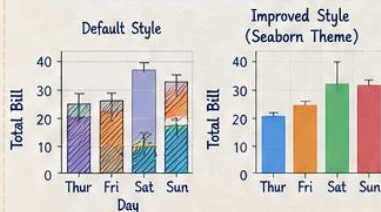


A good title and labels make your plot understandable at a glance.

4. Use Appropriate Aesthetics

- Choose a clean and consistent style.
- Use color palettes that are pleasant and accessible.
- Avoid too many colors.

```
sns.set_theme(style="whitegrid",
              palette="Set2",
              font_scale=1.1)
```

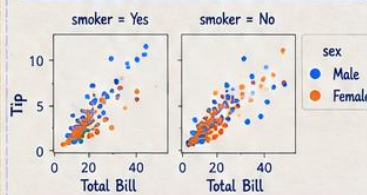


A clean style makes your visualizations look professional and easier to read.

5. Handle Multiple Plots Smartly

- Use FacetGrid, PairGrid or subplots to compare multiple views.
- Keep consistent scales, colors and labels.

```
g = sns.FacetGrid(df, col="smoker", hue="sex",
                  height=3)
g.map(sns.scatterplot, "total_bill", "tip")
g.add_legend()
```

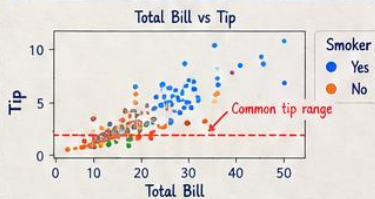


Multiple plots help you see patterns across different groups.

6. Highlight Insights, Not Just Plots

- Use annotations, reference lines or color highlights to emphasize key points.
- Remove unnecessary elements (chartjunk).

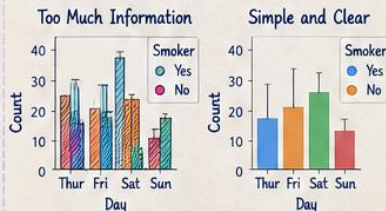
```
sns.scatterplot(data=df, x="total_bill", y="tip", hue="smoker")
plt.axhline(y=2.5, color="red", linestyle="--")
plt.annotate("Common tip range", xy=(30, 2.5),
            xytext=(32, 4), arrowprops=dict(arrowstyle="->", color="red"))
```



Guide your audience to the key takeaway.

7. Keep It Simple and Focused

- Avoid clutter (too many colors, labels, 3D effects, etc.).
- Show only what's relevant.
- Use white space effectively.



Less is more. A simple plot often communicates the message better.

8. Ensure Reproducibility

- Set a consistent theme and figure size.
- Save your plots with high resolution for reports or presentations.
- Use clear and organized code.

```
sns.set_theme(style="whitegrid")
plt.figure(figsize=(8, 5))
# your plot here
plt.tight_layout()
plt.savefig("seaborn_plot.png", dpi=300,
          bbox_inches="tight")
```

"Consistent and clean code today, better visualizations tomorrow!"

9. Key Takeaways

- Understand your data before plotting.
- Choose the right plot for your question.
- Use clear labels, titles and legends.
- Apply clean and consistent styling.
- Highlight key insights with annotations.
- Keep plots simple and focused.
- Use multiple plots to compare effectively.
- Ensure reproducibility (save and share).
- Always think about your audience.

"Great visualizations don't just show data, they tell a story!"

Learn Visualize Grow

DataXOdyssey

Same Data
Bigger Understanding!

Prepare
Practice
Visualize
Succeed!

Seaborn - Interview / Practical Cheat Sheet

Quick Reference for Common Questions, Code & Best Practices!

Same Concepts
Real Questions
Real Applications!

1. Load Data & Basic Setup

```
import pandas as pd
import seaborn as sns
import matplotlib.pyplot as plt

sns.set_theme(style="whitegrid",
              palette="Set2")
df = sns.load_dataset("tips")
df.head()
```

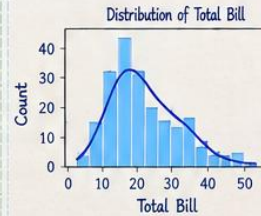
- Seaborn works directly with Pandas DataFrames.
- Use `sns.set_theme()` for consistent style.
- Common datasets: tips, iris, titanic, flights, penguins.

2. Common Plots & When to Use?

Plot	Function	Use Case
Scatter	<code>sns.scatterplot()</code>	Relationship between variables
Line	<code>sns.lineplot()</code>	Trends over time
Bar	<code>sns.barplot()</code>	Compare categories (with statistics)
Histogram	<code>sns.histplot()</code>	Distribution of single variable
KDE	<code>sns.kdeplot()</code>	Smooth distribution
Box	<code>sns.boxplot()</code>	Spread, outliers
Violin	<code>sns.violinplot()</code>	Distribution + density
Heatmap	<code>sns.heatmap()</code>	Correlation / matrix
Pairplot	<code>sns.pairplot()</code>	Multiple variables
Faceting	<code>sns.FacetGrid()</code>	Subplots by category

3. Distribution Plot (Histogram + KDE)

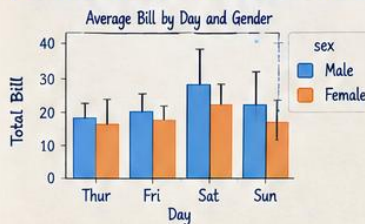
```
sns.histplot(df["total_bill"], kde=True,
             bins=20, color="skyblue")
plt.title("Distribution of Total Bill")
```



Use `histplot()` for distribution. Add `kde=True` to see the smooth curve.

4. Categorical Plot (Bar Plot)

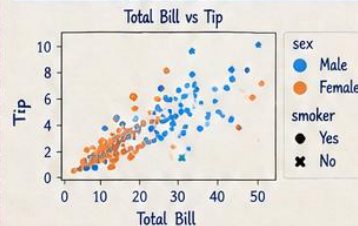
```
sns.barplot(data=df, x="day", y="total_bill",
            hue="sex", errorbar="sd",
            palette="Set2")
plt.title("Average Bill by Day and Gender")
```



Use `barplot()` for categorical comparisons. It shows the estimate (mean) with confidence intervals by default.

5. Relationship Plot (Scatter Plot)

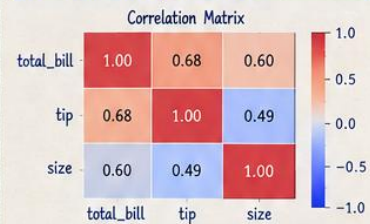
```
sns.scatterplot(data=df, x="total_bill",
               y="tip", hue="sex",
               style="smoker", s=80)
plt.title("Total Bill vs Tip")
```



Use `hue`, `style`, `size` to add more dimensions to your plot.

6. Heatmap (Correlation Matrix)

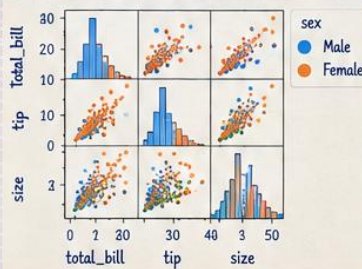
```
plt.figure(figsize=(6, 5))
sns.heatmap(df.corr(numeric_only=True),
           annot=True, cmap="cubehelix", fmt=".2f")
plt.title("Correlation Matrix")
```



Useful to quickly understand relationships between numeric variables.

7. Multi-Plot Grid (Pairplot)

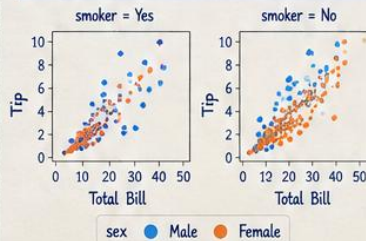
```
sns.pairplot(df, hue="sex",
             vars=["total_bill", "tip", "size"],
             palette="Set1")
```



Use `pairplot()` to explore relationships and distributions across multiple variables.

8. Faceting (FacetGrid)

```
g = sns.FacetGrid(df, col="smoker",
                  hue="sex", height=3)
g.map(sns.scatterplot, "total_bill", "tip")
g.add_legend()
```



Use `FacetGrid` to create subplots for different categories.

9. Common Customizations

- Change color palette → `sns.set_palette()`
- Add titles and labels → `plt.title()`, `plt.xlabel()`
- Adjust figure size → `plt.figure(figsize=(w, h))`
- Change markers → `style="o", "s", "x"`
- Remove grid → `sns.set_style("white")`
- Use log scale → `plt.xscale("log")`
- Save plot → `plt.savefig("plot.png", dpi=300)`

10. Quick Interview Questions

- What is the difference between `histplot()` and `kdeplot()`?
- When to use `barplot()` vs `countplot()`?
- How do you handle multiple variables in a plot?
- How can you customize the theme and style?
- How do you deal with outliers in visualizations?
- How do you interpret a heatmap?
- How is `FacetGrid` different from `pairplot()`?



Key Takeaways

- Seaborn + Pandas = simple, powerful and beautiful visualizations.
- Choose the right plot for your question.
- Use labels, titles and proper styling.
- Explore multiple variables with `pairplot()` or `FacetGrid()`.
- Practice with real datasets to build confidence.

DataXOdyssey

Practice these
and try with
your own data!

Visualize Practice Grow